STUDENT-COUNCIL ELECTION TALLY SYSTEM



PRESENTED BY:

1.) LAXMI SANKEERTANA K -2620080110

2.)SINDHUJA REDDY DEVA-2620080046



TOPIC OUTLINE

ABSTRACT

PROBLEM STATEMENT

A SIMPLE FLOW CHART

CO-1 CONCEPTS USED

CO-2 CONCEPTS USED

CO-3 CONCEPTS USED

CO-4 CONCEPTS USED

CO-5 CONCEPTS USED

CO-6 CONCEPTS USED

CONCLUSION

SCREEN SHOTS

GEO TAG PHOTO AND GITHUB LINK



YOUR VOICE MATTERS



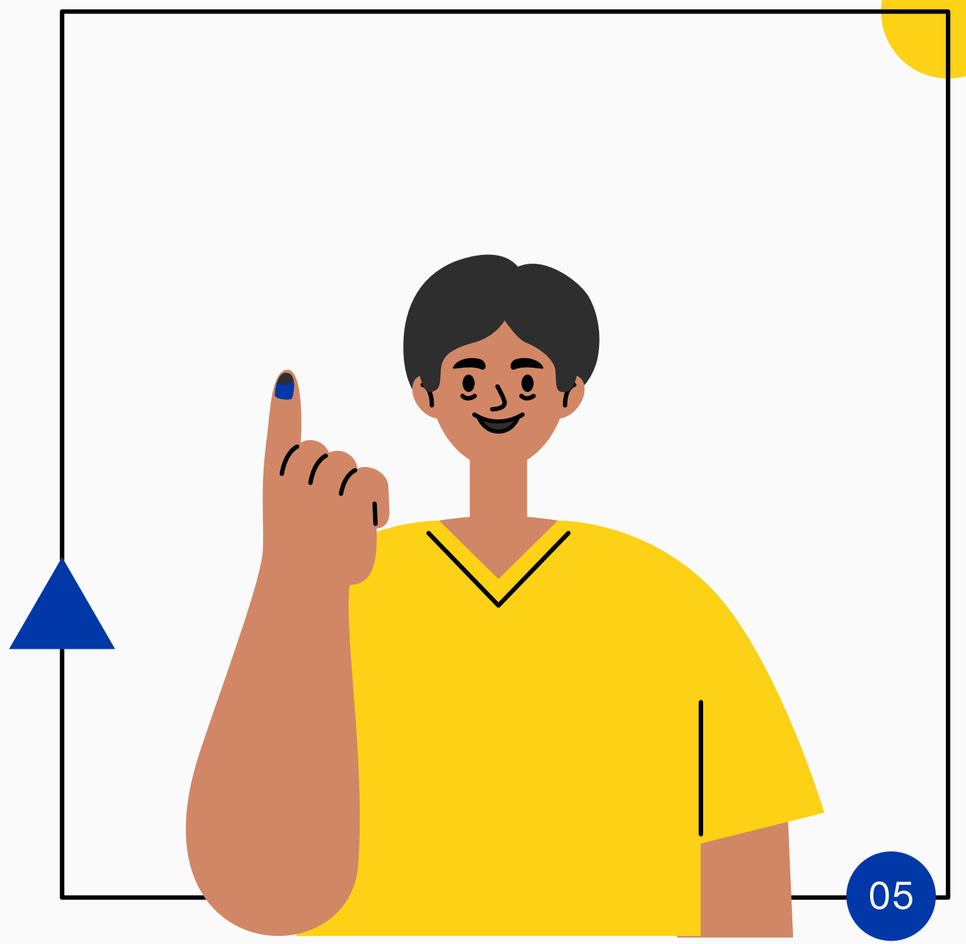
ABSTRACT

The Student-Council Election Tally System is a Java-based application created to make student elections simpler and more organized.

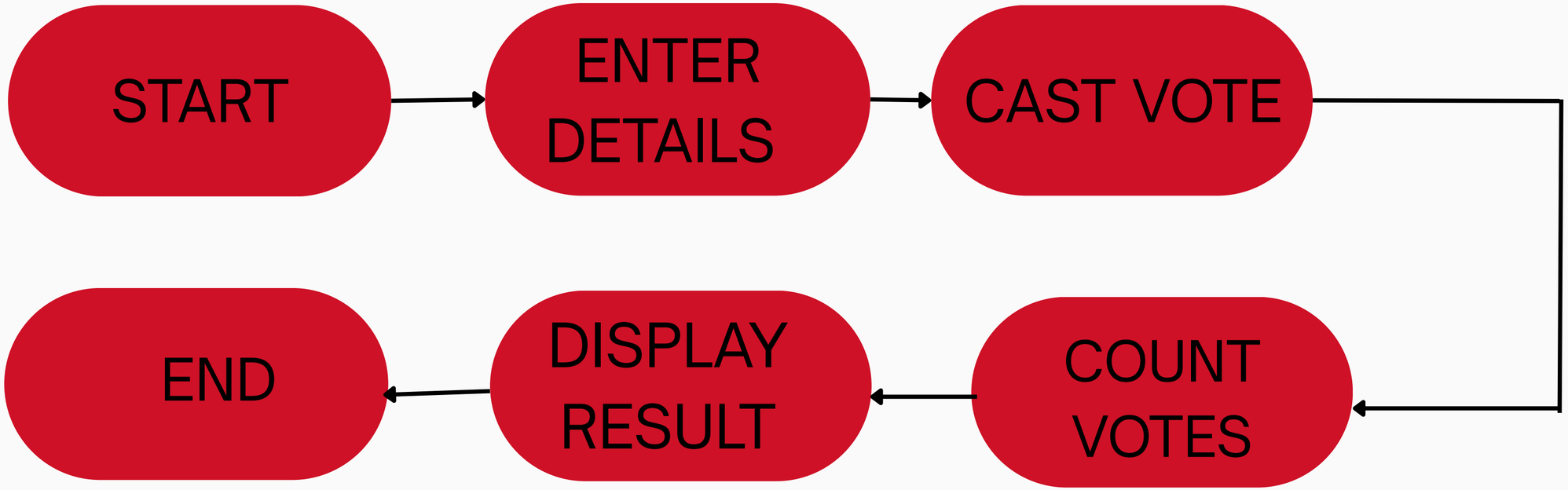
- It helps manage student and candidate details in one place.
- Votes can be recorded easily, while checking that a student does not vote more than once.
- The system automatically counts the votes received by each candidate.
- It displays the final results and helps identify the candidate with the highest number of votes.
- It reduces mistakes in manual counting and saves time during the election process.
- The project uses Java concepts such as classes, objects, conditions, loops, methods, and user input.
- In the future, more features like multiple council positions, voter verification, candidate search, and detailed reports can be added.

PROBLEM STATEMENT

Managing student-council elections manually can take a lot of time and may cause mistakes while recording student details, candidate details, and votes. Counting votes manually can also lead to errors, and there is a chance of students voting more than once if proper records are not maintained. To solve these issues, we developed a simple Java-based system that organizes the election process, records votes, prevents duplicate voting, counts votes accurately, and displays the final results.



A SIMPLE FLOWCHART



C0-1 CONCEPTS USED

SNO	CONCEPT	APPLICATION
1.)	Java Structure	public class StudentCouncilElection and main() method
2.)	Primitive Data Types & Variables	int n, m, int studentId[], int voteCount[]
3.)	String	String candidateName[] for storing candidate names
4.)	Arithmetic Operators	choice - 1 and (i + 1)
5.)	Relational Operators	i < n, i < m, choice >= 1, choice <= m
6.)	Logical Operators	choice >= 1 && choice <= m for validating the vote
7.)	Assignment Operators	max = voteCount[i], index = i
8.)	Scanner/Console Input	Scanner sc = new Scanner(System.in) and sc.nextInt(), sc.nextLine()

C0-2 CONCEPTS USED

SNo	CONCEPT	APPLICATION
1.)	For Loop	Used to enter candidate names, take votes, and display candidates/results
2.)	Nested For Loop	Used inside voting() to display all candidate names for each student
3.)	If-else	Used to check whether the entered candidate choice is valid
4.)	Relational Operators	choice >= 1 && choice <= m for checking valid choices
5.)	Logical Operator	&& combines both conditions during vote validation

C0-3 CONCEPTS USED

SNO	CONCEPTS	APPLICATION
1.)	Methods	enterCandidates(), voting(), and displayResult() divide the program into smaller tasks
2.)	Method Invocation	Methods are called from main() to execute each part of the election process
3.)	1D Arrays	studentId[], candidateName[], and voteCount[] store project data
4.)	Array Traversal	for loops are used to access candidate and vote information
5.)	Counting Technique	voteCount[choice - 1]++ increases the vote count for the selected candidate
6.)	Searching/Comparision	displayResult() compares vote counts to find the candidate with the highest votes
7.)	Parameters	Methods use project data such as candidate and voting information

C0-4 CONCEPTS USED

SN0:	CONCEPT	APPLICATION
1.)	Classes and Objects	Person, Student, Candidate
2.)	Object Creation	new Candidate(name) and new Student(...)
3.)	Parameterized Constructors	Candidate(String name), Student(int studentId, String name)
4.)	Inheritance	class Student extends Person
5.)	Encapsulation	Private fields name and voteCount in Candidate

C0-4 CONCEPTS USED

6.)	Accessor Methods	getName() and getVoteCount()
7.)	Method Overriding	Student overrides displayName()
8.)	Object Representation	Candidate overrides toString() to display results
9.)	Static Members	static Scanner sc, static methods
10.)	Exception Handling	try-catch in readNumber()
11.)	Unchecked Exception	NumberFormatException is caught when input is not a valid integer

C0-5 CONCEPTS USED

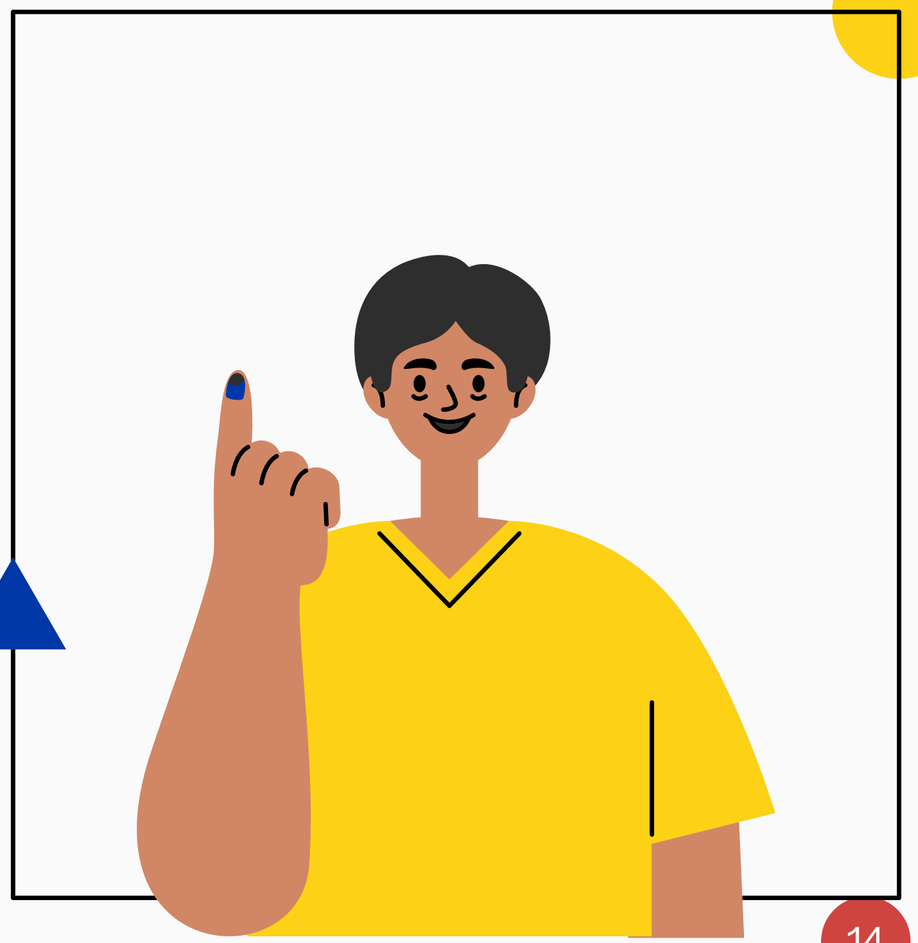
SNO:	CONCEPT	APPLICATION
1.)	String Abstraction	Candidate names and input messages use String
2.)	String Trimming	sc.nextLine().trim() removes leading and trailing whitespace
3.)	String Validation	name.isEmpty() prevents empty candidate names
4.)	String Concatenation	"Voter " + (i + 1) creates voter labels
5.)	Text Parsing	Integer.parseInt(...) converts text input into an integer
6.)	String Immutability	String operations produce new strings rather than modifying the original
7.)	Object-to-String Conversion	toString() returns a readable candidate result

C0-6 CONCEPTS USED

SNO:	CONCEPT	APPLICATION
1.)	ArrayList	Stores candidates
2.)	Generics	ArrayList<Candidate>
3.)	Lambda Expressions	Filters candidates
4.)	Stream API	Processes results
5.)	filter() and count()	Counts candidates receiving votes
6.)	sorted() and Comparator	Ranks candidates by votes
7.)	Method References	Displays results
8.)	collect()	Collects winning candidates

CONCLUSION

- We built a working Java-based system that automates student council elections end-to-end.
- It solves real problems — duplicate voting and manual counting errors — using classes, arrays, loops, and methods.
- The project can be extended with features like voter verification and multiple positions in the future



GEO TAG PHOTO AND GITHUB LINK

THANK YOU FOR LISTENING!

